Q13. Psychological Creativity as a Special Function of Historical Creativity

> **Question.** Model psychological (P) creativity as a special function of historical (H) creativity. Begin with a constrained search space so most actions can be creative. Slowly expand the space and eliminate states. When no state is left, combine the n top-performing states into new states. Continue until no new state falls beyond the final expanded space, then deliver the final states ranked (Diagram 5).

1. The idea in plain words

H-creativity means the idea is new to the whole world. **P-creativity** means it is new *to me*, even if others had it before. The only difference is **what memory we compare against**. H compares with everything humans have ever produced. P compares only with what this agent has seen. So P is H with a smaller memory. Because the agent knows less than history, anything that is new to history is also new to the agent. Every H-creative idea is P-creative, but not the other way round.

The algorithm then works like a curious person exploring: start with a small, safe area near one idea (so almost everything feels new), widen it slowly, throw out weak ideas, and when the area is used up, **mix the best ideas to make new ones** and move on.

2. Modelling P as a special function of H

Measure	Definition
$\text{novelty}_H(s)$	min distance from idea s to any idea in the **history archive**
$\text{novelty}_P(s)$	min distance from s to any idea in the **agent archive**, where $\text{Archive}_{agent} \subseteq \text{Archive}_{history}$
H-creative	$\text{novelty}_H(s) \geq \tau$ and value(s) is high
P-creative	$\text{novelty}_P(s) \geq \tau$ and value(s) is high
Special-case rule	**$P(s) = H(s \mid \text{archive} = \text{agent's own memory})$. Since the agent archive is a subset, $\text{novelty}_P \geq \text{novelty}_H$ always.**

3. Assumptions

- An idea is a point in a 2-D conceptual space (0 to 11.5 on each axis). Its **value** (usefulness, 0 to 100) comes from a landscape with one broad slope and three hills.
- The agent starts at idea **j = (2, 2)**, which has value 12.
- A pool of 260 candidate ideas is spread over the space. The history archive has 30 ideas and the agent remembers only 7 of them.
- Novelty threshold $\tau = 1.0$. Ideas with value below 14 are weak and are eliminated. n = 4 top states are combined.
- The search circle starts at radius 1, grows by 1 per round, and is capped at 3 per stage.

4. Step-by-step approach

1. **Constrain:** start with a small circle (r = 1) around j. Few states, so most actions are new to the agent.

2. **Examine and eliminate:** visit every unvisited state in the circle. Remove visited states from the space and eliminate those with low value. Keep the good ones.
3. **Expand slowly:** grow r by 1 and repeat, until the cap is reached and no unvisited state is left.
4. **Combine:** take the n best kept states and blend each pair into a new state (a point on the line between them, a little beyond if lucky).
5. **Move the centre** to the best new state (j → n_1 → n_2 ...) and start a new small circle there.
6. **Stop** when none of the new states lies beyond the space already covered. Then rank all states by value.

Constrained space -> expand -> eliminate -> combine

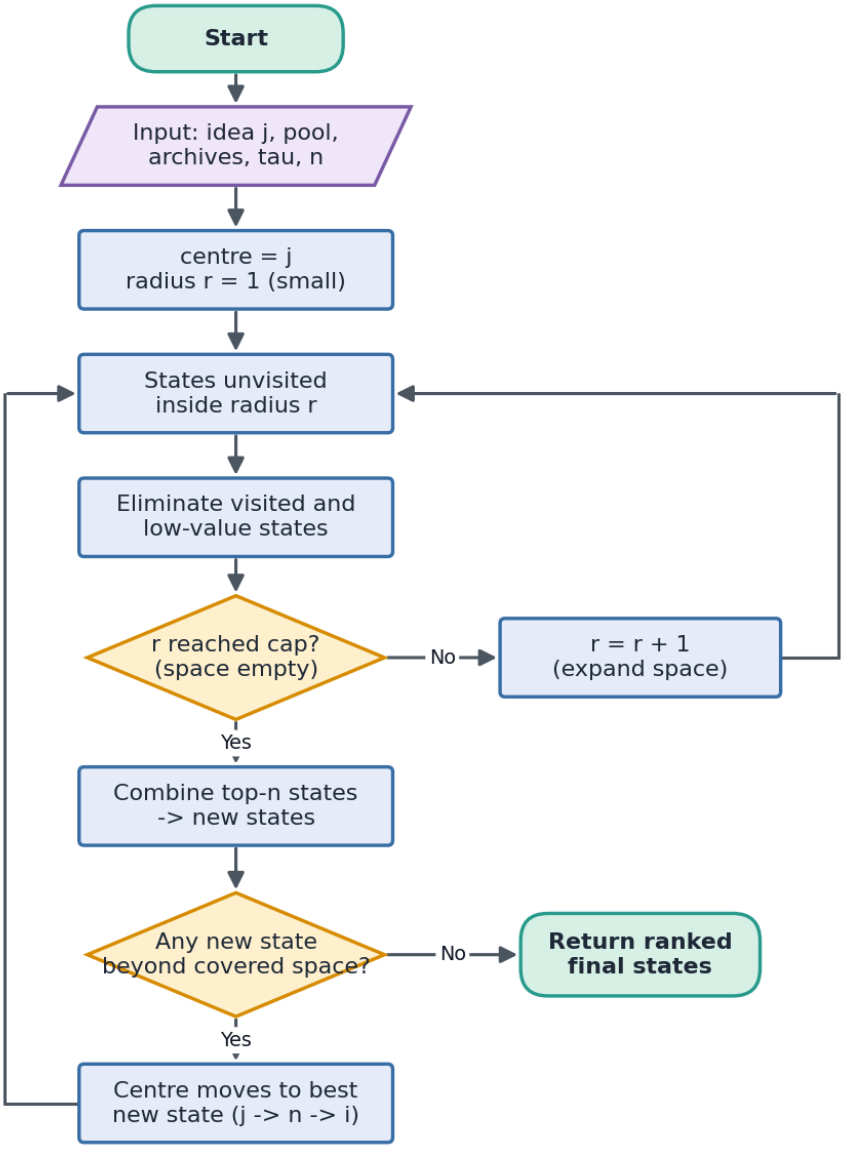


Figure 1. Flowchart of the constrained-then-expanding search.

5. Pseudocode

```
function CreativeSearch(pool, j, tau_v, n):
    centre ← j;  VISITED ← { };  KEPT ← [ ];  COVERED ← [ ]
    repeat (stage = 1, 2, ...):
        r ← r0                                          # small, constrained circle
        loop:
            SPACE ← { s in pool : s not in VISITED, dist(s, centre) ≤ r }
            KEPT += { s in SPACE : value(s) ≥ tau_v }    # weak states are eliminated
            VISITED += SPACE                                # visited states are eliminated
            if r ≥ r_cap: break                          # space is empty
            r ← r + dr                                  # slow expansion
        COVERED.add(circle(centre, r_cap))
        TOP ← n best of KEPT by value
        NEW ← { blend(a, b) : a, b in TOP }             # combine top performers
        pool += NEW
        if no state of NEW is outside COVERED: break
        centre ← best of NEW                            # j -> n -> i
    return all states ranked by value
```

6. Worked example (real output)

Stage	Radius	States in space	Kept	Eliminated	New states created
1	1	7	2	5	0
1	2	22	6	16	0
1	**3**	**21**	**15**	**6**	**6**
2	1 / 2 / 3	5 / 15 / 26	5 / 15 / 26	0	6 (at r = 3)
3	1 / 2 / 3	7 / 13 / 21	7 / 13 / 21	0	6 (at r = 3)

At the start the circle around j holds only weak ideas (value around 12), so 5 of 7 are eliminated. By the end of stage 1, the circle is empty and the 4 best kept states are blended into 6 new ones. The centre jumps from j to **(4.1, 4.5)**. In stage 2 the circle is already in a better area, so nothing is eliminated and the next jump goes to **(3.4, 7.9)**. In stage 3 the six new states all fall inside the space already covered, so the search stops.

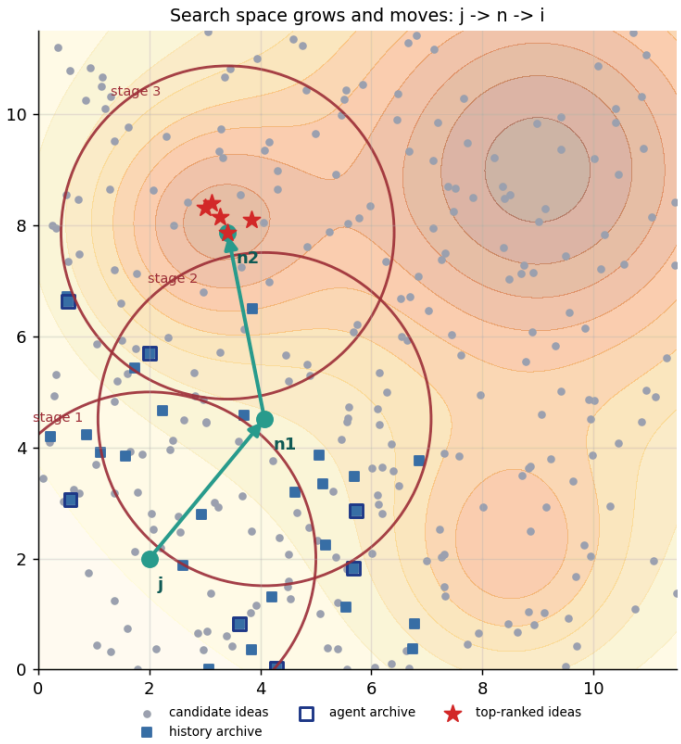


Figure 2. Diagram 5 with real data. Red circles are the search spaces of stages 1 to 3. The green arrow is the path $j \to n_1 \to n_2$. Red stars are the top-ranked ideas. Shading shows value.

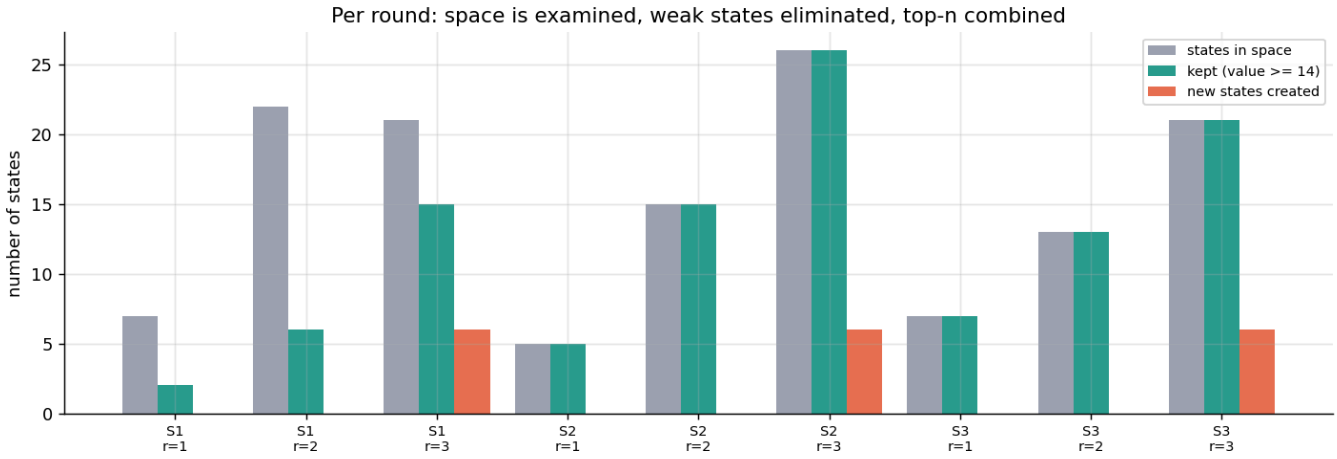


Figure 3. Counts per round. Elimination is heavy in stage 1 and disappears once the centre moves to a good area. New states are created only when a stage ends.

7. Final ranked set

Idea (x, y)	Value	novelty_P	novelty_H	P-creative	H-creative
(3.3, 8.2)	85.3	2.76	1.76	yes	yes
(3.4, 7.9)	85.0	2.59	1.44	yes	yes
(3.0, 8.3)	83.5	2.81	2.01	yes	yes
(3.1, 8.4)	83.4	2.93	2.05	yes	yes
(3.8, 8.1)	83.1	3.03	1.60	yes	yes

The start idea j had value 12, and the best final idea has value 85.3. Across all 117 states the search kept or created, **97 are P-creative and 53 are H-creative, and every H-creative one is also P-creative**. Figure 4

shows why: no point sits above the diagonal.

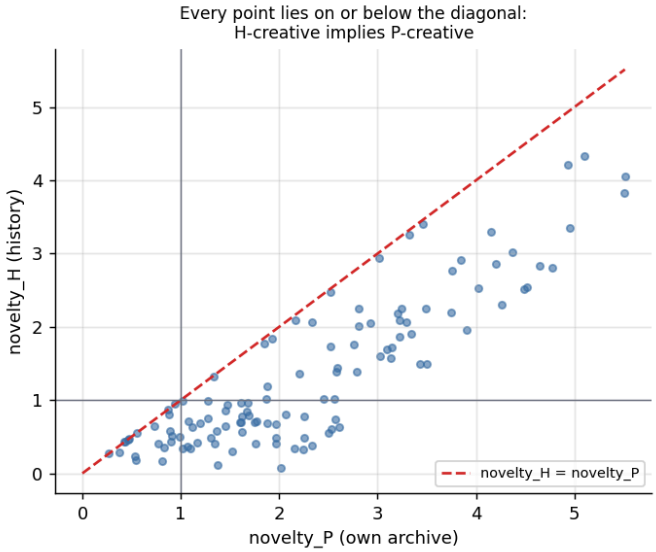


Figure 4. novelty_H against novelty_P for all states. H never exceeds P.

> **Honest limit.** The search ended on the hill at (3, 8) with value 85. The tallest hill (around (9, 9), value about 100) was never reached, because every new state was built from states near the old ones. Restarting from a second idea, or letting the combination step extrapolate further, would reach it. This is the same local-maximum problem as hill climbing.

8. Why start constrained?

A small first circle keeps the agent's decisions simple and cheap, and it makes most moves new to this agent (P-creative) because it has seen so little. Constraints also focus creativity, as the course notes say. Widening the circle slowly, and only combining when the space is empty, is how the agent moves from *exploring* to *combining* and toward H-creativity. It also corresponds to Boden's forms: exploring inside the space first, then combinational creativity, and a shifting centre that gradually transforms the space.

9. Code and real run

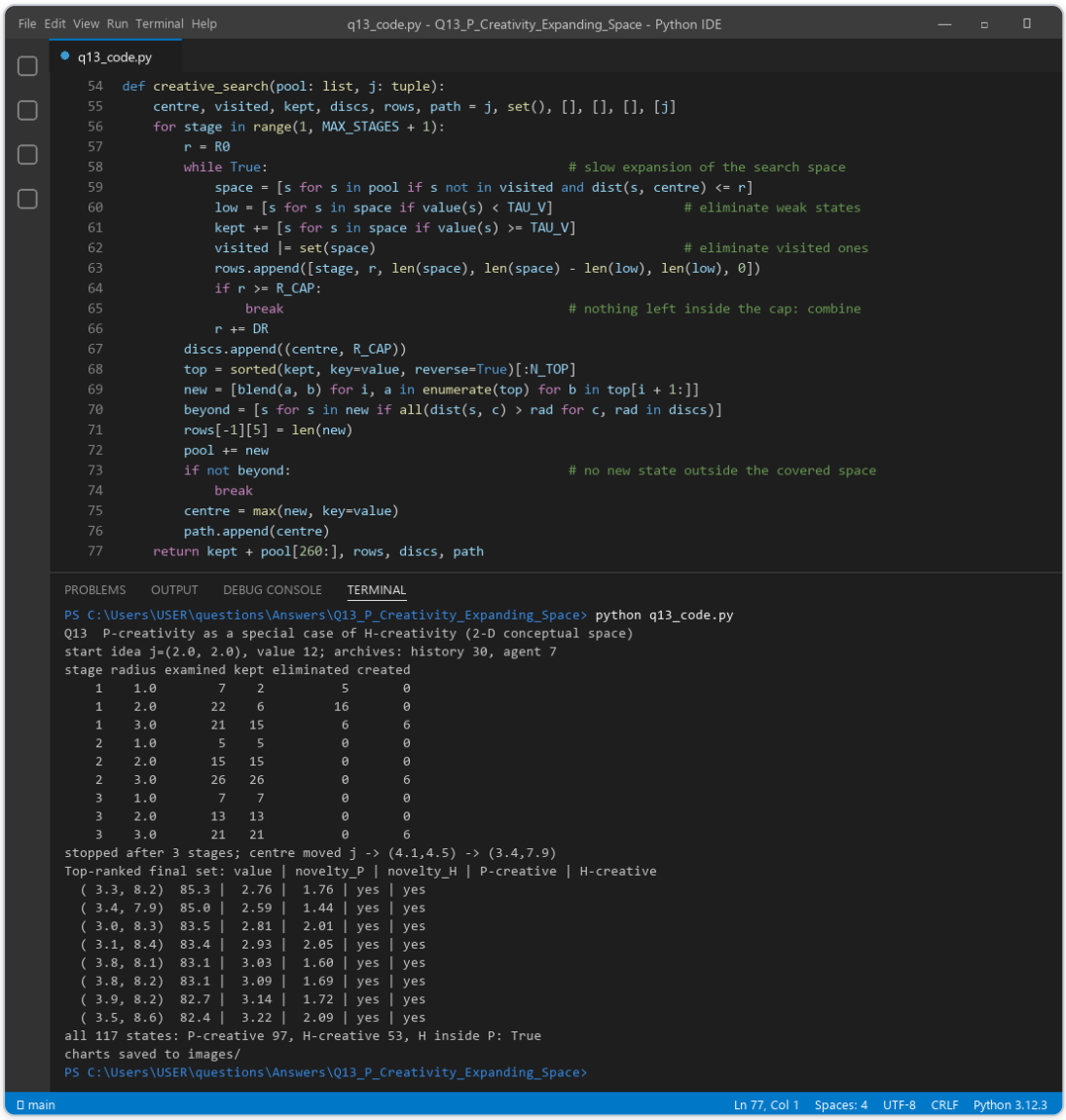


Figure 5. The creative_search function (lines 54-77) and the real terminal output.

10. Summary

> P-creativity is H-creativity measured against a smaller memory: $\text{novelty}_P \geq \text{novelty}_H$, so every H-creative idea is P-creative. The algorithm starts in a small circle, expands it slowly, eliminates visited and weak states, and when the space is empty combines the top n states, moving the centre j → n → i until no new state lies outside the covered space. In the real run, the best idea improved from value 12 to 85.3, with 53 H-creative states among the 117 produced.

Files in this folder: `q13_code.py` (source), `q13_code.ipynb` (Colab/Jupyter version, already run), `q13_output.txt` (console output), `images/` (all figures).